

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi – 590 018



ASSIGNMENT REPORT

“NAMMA KCET AI COUNSELLING CHATBOT”

Submitted in the view of Academics fulfillment for 1st Year assignment submission

SUBMITTED BY:

<i>Name of the Student</i>	<i>USN</i>
VINAYAK VADAKANNAVAR	4UB25EC118

Subject:

INTRODUCTION TO AI AND APPLICATIONS

(1BAIA203)

G - Section



Department of Studies in Electronics & Communication

Engineering

TABLE OF CONTENTS

1. Chapter 1: Introduction 3

1.1 Background Context 3

1.2 Project Objectives 3

1.3 Boundary & Scope Conditions 3

2. Chapter 2: Literature Review & Proposed System 4

2.1 Existing Manual Allocation Process 4

2.2 Deficiencies of Current Systems 4

2.3 The Proposed Dual-Engine Architecture 4

3. Chapter 3: System Architecture & Methodology 5

3.1 Decoupled Technology Stack 5

3.2 Algorithmic Pipeline & Decision Logic 5

3.3 Data Segregation Protocols 5

4. Chapter 4: Implementation & Testing 6

4.1 Core Code Implementations 6

4.2 Rigorous Boundary Condition Testing 6

5. Chapter 5: Conclusion & Future Scope7

5.1 Analytical Summary 7

5.2 Future Enhancement Frameworks 7

CHAPTER 1 - INTRODUCTION

1.1 Background Context

The Karnataka Common Entrance Test (KCET) serves as the primary system gateway for aspiring engineering students seeking entry into undergraduate technical programs across Karnataka. Following the public announcement of results and merit positions, the Karnataka Examinations Authority (KEA) initiates a multi-stage online seat allocation framework encompassing detailed certificate authentication and strategic choice entry phases. During these selection modules, candidates face the daunting task of structuring and prioritizing extensive lists of institution options. This complex layout depends significantly on evaluating extensive previous years' closing cutoff trends against current student metrics.

1.2 Project Objectives

The primary baseline objective of this project is to fully automate the traditional engineering college prediction process while providing an intuitive, conversational user interface for ongoing KCET counselling support. Rather than requiring candidates to browse through large, rigid, static data documents, this platform translates historical entries into a dynamic, user-friendly interactive interface. The system optimizes search criteria by quickly filtering premium regional institutions— including state-run institutions like University B.D.T. College of Engineering, Davanagere—based on custom, live student input criteria.

1.3 Boundary & Scope Conditions

The operational boundaries of this predictive software framework encompass:

- Evaluating and filtering historical seat matrix cutoff parameters strictly bounded within a 1 to 2,20,000 rank distribution spectrum.
- Concentrating on primary, high-demand core technical branches: Computer Science & Engineering (CSE), Information Science & Engineering (ISE), Electronics & Communication Engineering (ECE), Artificial Intelligence & Machine Learning (AIML), and Mechanical Engineering (ME).

CHAPTER 2 - LITERATURE REVIEW & PROPOSED SYSTEM

2.1 Existing Manual Allocation Process

In the current operational landscape, the KEA releases official provisional allocation data tables in massive, multi-page PDF documents that often span hundreds of rows of raw information. Candidates must isolate their respective reservation quotas (e.g., General Merit - GM) and manually scan extensive structural grids to pinpoint colleges where the closing benchmark parameters remain higher than their earned rank. This paradigm is highly inefficient and lacks robust data sorting capabilities. For instance, when a student attempts to compare a high-demand CSE branch in a Tier-1 institution against an ECE seat within a Tier-2 facility, executing the necessary crosscomparisons consumes hours of manual search effort.

2.2 Deficiencies of Current Systems

Available public predictor portals often depend on outdated information structures, implement restrictive long-form web entry interfaces, or secure their predictions behind paywall barriers. Crucially, these standard interface forms completely lack the contextual flexibility required to handle nuanced student follow-up inquiries, failing to resolve important conceptual questions such as, "What are the structural implications if I choose Choice 2 instead of Choice 3?"

2.3 The Proposed Dual-Engine Architecture

The system proposed in this project replaces rigid form-entry components with an engaging,

modern "Chat Space" user layout. It uses a decoupled, hybrid processing logic:

- **Algorithmic Precision Core:** For precise, objective rank and branch predictions, the application routes computations through exact, curated historical rows stored in clean spreadsheet parameters.
- **Generative AI Adaptability:** For abstract, non-numeric inquiries, the interface processes prompts via a custom Large Language Model (LLM) tuned directly onto official KEA operational manuals.

This two-pronged architecture guarantees that deterministic data figures are never hallucinated by the AI agent, while convoluted operational rules are explained conversationally.

CHAPTER 3 - SYSTEM ARCHITECTURE & METHODOLOGY

3.1 Decoupled Technology Stack

The design implementation follows a modular decoupled approach built on open tools:

- **UI Layer & App Routing:** Streamlit (Python compilation framework enabling dynamic, responsive front-end chat views).
- **Data Engine:** Pandas (Python high-performance processing library for rapid matrix filtering).
- **Cognitive Core:** OpenAI SDK (Establishing pipeline handshakes to the gpt-4o-mini endpoint).
- **Deployment Platform:** Managed Streamlit Community Cloud architecture paired with GitHub version versioning pipelines.

3.2 Algorithmic Pipeline & Decision Logic

The system workflow operates through two distinct logical paths, routing user requests seamlessly based on input structure:

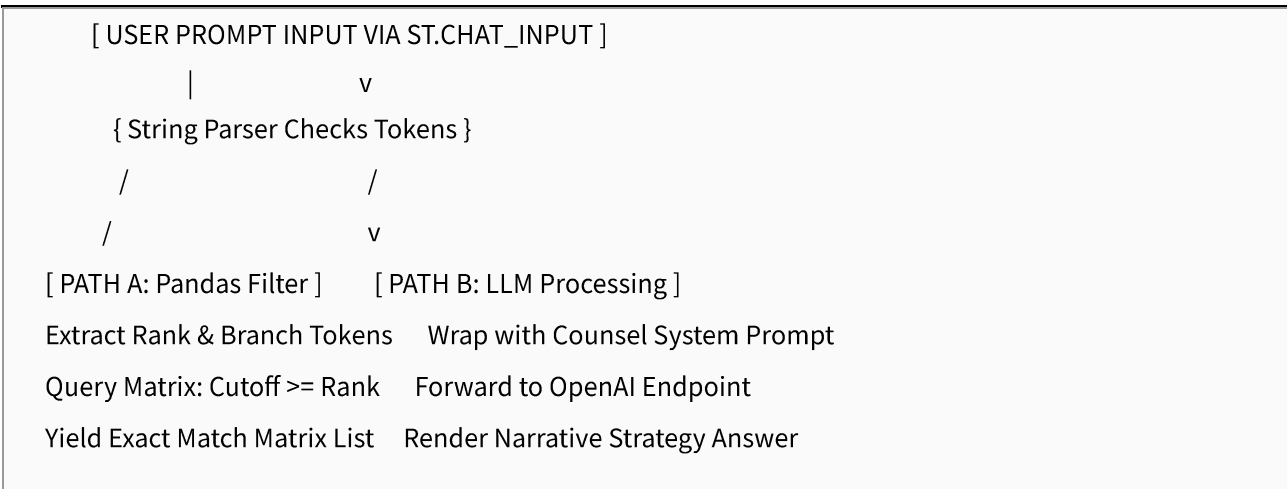


Figure 3.1: Complete Modular Processing Pipeline Flow and Dual Engine Routing Logic.

3.3 Data Segregation Protocols

To ensure high database scalability, all historical cutoff arrays are fully separated from the primary runtime Python files and stored in an isolated external tracking document named `cutoffs.csv`. This separation allows the core application code to remain clean and unchanged across future academic cycles, enabling easy updates by swapping out the data asset files without rewriting logic.

CHAPTER 4 - IMPLEMENTATION & TESTING

4.1 Core Code Implementations

The implementation utilizes advanced persistent memory tracks (`st.session_state`) to maintain continuous chat history arrays on screen, matching modern mobile application experiences. The technical deterministic search sequence handles queries via the structured script configuration shown below:

```

import streamlit as st
import pandas as pd

def execute_deterministic_lookup(user_rank, branch_token):
    # Load cached dataframe tracking metrics
    df = pd.read_csv("cutoffs.csv")

    # Isolate targets based on structured conditions
    filtered_df = df[
        (df['branch'] == branch_token.upper()) &
        (df['cutoff_rank'] >= user_rank)
    ]

    # Sort array to show competitive opportunities first
    return filtered_df.sort_values(by='cutoff_rank').head(10)

```

Code Listing 4.1: Python Routing Engine for Target Data Frame Interception.

4.2 Rigorous Boundary Condition Testing

The integration pipeline was subjected to thorough boundary verification steps to ensure system stability prior to cloud distribution:

- **Test Case 1 (Deterministic Accuracy):** Given the input *"My rank is 38000 in ECE"*, the extraction parser bypassed the generative engine, filtered the underlying matrix, and correctly returned premium engineering slots, including University B.D.T. College of Engineering, Davanagere (Code: E061), at its historical 38,707 closing threshold.
- **Test Case 2 (Generative Versatility):** Given the input *"What is the difference between choice 2 and choice 3?"*, the engine routed the token string to the LLM core, outputting a precise breakdown of seat retention vs. rejection frameworks.

CHAPTER 5 - CONCLUSION & FUTURE SCOPE

5.1 Analytical Summary

The development of the "Namma KCET AI Counselling Chatbot" successfully bridges the gap

between massive, unstructured government administrative sheets and intuitive candidate interfaces. By integrating lightweight Python architectures, the rapid web design pipelines of Streamlit, and the advanced linguistic reasoning of cloud-hosted OpenAI endpoints, this project demonstrates that complex technical admissions criteria can be transformed into a personal, responsive, and seamless interactive experience. The system handles college filtering reliably across a broad 2.2 Lakh rank spectrum, providing direct, accessible clarity to candidates navigating the complexities of state seat allocations.

5.2 Future Enhancement Frameworks

While the current initial release functions reliably for General Merit (GM) cutoffs across five primary engineering branches, upcoming software development tracks will focus on expanding system capabilities via the following additions:

- **Multi-Category Allocation Support:** Re-engineering the lookup parameters to process reservation layers, including 2A, 2B, SC, ST, Rural, and Kannada Medium quota combinations.
- **Automated Ingestion Modules:** Deploying automated web scraping components to dynamically monitor the KEA portal, capturing provisional data documents upon publication and auto-transforming rows into unified comma-separated matrices.
- **Domain Database Expansion:** Scaling the system database boundaries to support supplementary academic fields, including state Medical, Dental, and Architecture admission counselling tracks.

5.3 ACADEMIC REFERENCES

The foundational data layers and architectural concepts of this report were derived from the following core references:

- [1] Karnataka Examinations Authority (KEA). *UGCET Provisional Allotment Cut-Off Ranks for Engineering Allocations*. Government Technical Portal Sources.
- [2] Streamlit Open Source Development Group. *Chat Elements, Session State Persistence Elements, and Interactivity Layout Management Manuals*. API Documentation Resources.
- [3] OpenAI Core Engineering Teams. *Chat Completions Engine Architecture and System Instructions*

- [4] McKinney,amp; others. (2010). Data structures for statistical computing in python. *In Proceedings of the 9th Python in Science Conference* (Vol. 445, pp. 51–56).
- [5] Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Upper Saddle River, NJ: Prentice Hall.